

JUPYTER NOTEBOOK





Jupyter Notebook – графічна вебоболонка для IPython для інтерактивних обчислень та спільної розробки додатків (IPython підтримує паралельні підключення до одного обчислювального ядра).

Особливості Jupyter Notebook:

- збереження історії введення у сеансах, логування сесії;
- кешування вихідних результатів;
- графічне представлення результатів;
- підтримка мови розміток Markdown і LaTeX;
- розширювана система магічних команд.

Компоненти Jupyter Notebook:

- браузерний вебзастосунок;
- ноутбуки – файли для роботи з кодом програми.

Ноутбук – файл, у якому зберігається код та отримані в межах сесії вхідні та вихідні дані. Він є документованим записом роботи з можливістю багаторазового запуску частин коду. Ноутбук можна експортувати в формати PDF і HTML.



Встановлення та запуск

Jupyter Notebook входить до складу Python-дистрибутиву Anaconda. Процес встановлення та запуску описаний на офіційному сайті anaconda.com та на багатьох інших онлайн-ресурсах.



Основи роботи в оболонці

← → ↻ 🏠 ⓘ localhost:8888/tree ☆ ✉ 📅 ⚙️ 👤 ⋮

 jupyter Quit Logout

Files Running Clusters

Select items to perform actions on them. Upload New ▾ ↻

☐ 0 ▾ 📁 /	Name ▾	Last Modified	File size
☐ 📁 anaconda3		5 місяців тому	
☐ 📁 cpp_projects		11 днів тому	
☐ 📁 Documents		місяць тому	
☐ 📁 PlayOnLinux's virtual drives		рік тому	



Створення ноутбука та ін. файлів

The image shows the JupyterLab web interface. At the top, there's a header with the Jupyter logo and the word 'jupyter'. On the right side of the header are 'Quit' and 'Logout' buttons. Below the header, there are three tabs: 'Files', 'Running', and 'Clusters'. The 'Files' tab is active. Below the tabs, there's a message: 'Select items to perform actions on them.' Below this message is a file browser view showing a list of files and folders. The first item is a folder icon followed by a dropdown arrow and a slash '/'. Below this are four items, each with a checkbox and a folder icon: 'anaconda3', 'cpp_projects', 'Documents', and 'PlayOnLinux's virtual drives'. On the right side of the file browser, there are buttons for 'Upload', 'New', and a refresh icon. The 'New' button is clicked, and a dropdown menu is open. The menu has two sections: 'Notebook:' with a text input field containing 'Python 3', and 'Other:' with three options: 'Text File', 'Folder', and 'Terminal'.

jupyter

Quit Logout

Files Running Clusters

Select items to perform actions on them.

0 /

anaconda3

cpp_projects

Documents

PlayOnLinux's virtual drives

Upload New

Notebook:

Python 3

Other:

Text File

Folder

Terminal



Виконання коду

Ctrl+Enter виконує код:

```
In [1]: 2**10
Out[1]: 1024
```

Shift+Enter виконує код і створює нову комірку, у якій можна використовувати дані попередньої:

```
In [11]: x = 2**10
         print(x)
         1024

In [12]: x *= 2
         print(x)
         2048
```



Керування комірками

The screenshot displays the Jupyter Notebook interface. At the top, there is a menu bar with options: File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. Below the menu bar is a toolbar with icons for saving, adding a new cell, deleting a cell, copying, pasting, and navigating between cells. The main area contains two code cells. The first cell, labeled 'In [11]:', contains the code `x = 2**10` and `print(x)`, with the output `1024` displayed below it. The second cell, labeled 'In [12]:', contains the code `x *= 2` and `print(x)`, with the output `2048` displayed below it. A context menu is open over the 'Cell' menu item, showing options: Run Cells, Run Cells and Select Below, Run Cells and Insert Below, Run All, Run All Above, Run All Below, Cell Type, Current Outputs, and All Output.

File Edit View Insert Cell Kernel Widgets Help

Run Cells
Run Cells and Select Below
Run Cells and Insert Below
Run All
Run All Above
Run All Below
Cell Type
Current Outputs
All Output

```
In [11]: x = 2**10  
         print(x)  
1024
```

```
In [12]: x *= 2  
         print(x)  
2048
```



“Магічні” команди

```
In [2]: %lsmagic
```

```
Out[2]: Available line magics:
```

```
%alias %alias_magic %autoawait %autocall %automagic %autosave %bookmark %cat %cd %clear %colors %conda  
%config %connect_info %cp %debug %dhist %dirs %doctest_mode %ed %edit %env %gui %hist %history %killb  
%gscripts %ldir %less %lf %lk %ll %load %load_ext %loadpy %logoff %logon %logstart %logstate %logstop  
%ls %lsmagic %lx %macro %magic %man %matplotlib %mkdir %more %mv %notebook %page %pastebin %pdb %pde  
%f %pdoc %pfile %pinfo %pinfo2 %pip %popd %pprint %precision %prun %psearch %psource %pushd %pwd %pyc  
%at %pylab %qtconsole %quickref %recall %rehashx %reload_ext %rep %rerun %reset %reset_selective %rm %r  
%mdir %run %save %sc %set_env %store %sx %system %tb %time %timeit %unalias %unload_ext %who %who_ls  
%whos %xdel %xmode
```

```
Available cell magics:
```

```
%%! %%HTML %%SVG %%bash %%capture %%debug %%file %%html %%javascript %%js %%latex %%markdown %%perl %  
%prun %%pypy %%python %%python2 %%python3 %ruby %script %sh %svg %sx %system %%time %%timeit %%  
writefile
```

```
Automagic is ON, % prefix IS NOT needed for line magics.
```

Документація з магічних команд: <https://ipython.org/ipython-doc/3/interactive/magics.html>

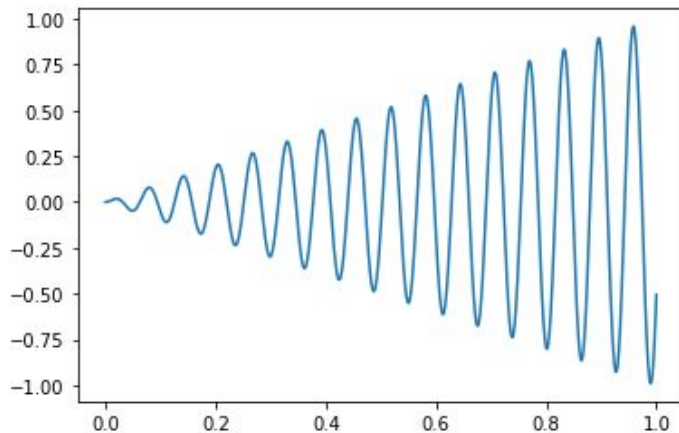


У деяких версіях за замовчуванням зображення не виводяться у ноутбучі. Для їх відображення потрібно виконати спеціальну команду, яка починається символом %. Наприклад, для бібліотеки matplotlib:

```
In [6]: %matplotlib inline
from matplotlib import pyplot as plt
from math import sin

x = [i/1000 for i in range(1001)]
y = [i*sin(100*i) for i in x]

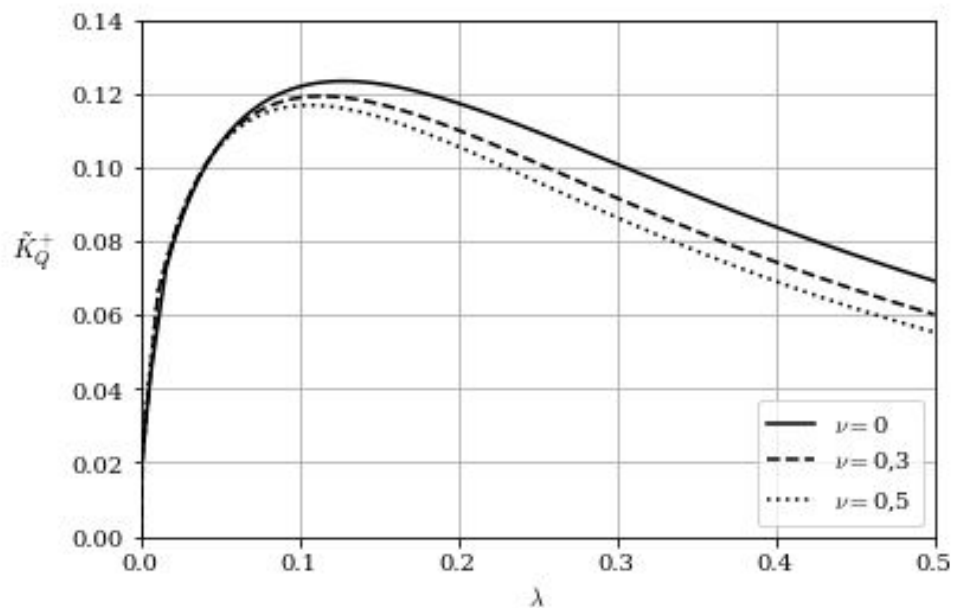
plt.plot(x, y)
plt.show()
```





Виконання скрипта .py та інших ноутбуків

```
In [1]: %run ./draw_graph.py
```





Час виконання коду в комірці

```
In [4]: %%time
```

```
x = 2**100000
```

```
CPU times: user 445 µs, sys: 67 µs, total: 512 µs
```

```
Wall time: 520 µs
```

`%timeit` запускає код багато разів (кількість можна задати) та виводить середній час з кількох найшвидших виконань:

```
In [7]: %timeit x = 2**100000
```

```
501 µs ± 10 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```



Супровід коду (розмітка)

Диференціальне рівняння:

```
\begin{align} \dot{x} &= \sigma(y-x) \\ \end{align}
```

Результат:

Диференціальне рівняння:

$$\dot{x} = \sigma(y - x)$$



Віджети

```
import ipywidgets as widgets  
from IPython.display import display
```

```
slider = widgets.IntSlider(min = 3) # min = 0, max = 100, step = 1, value = 0, ...  
display(slider)
```



Значення зберігається в атрибуті value:

```
slider.value
```

52

Атрибути віджетів можна змінювати:

```
slider.value = 50  
slider.orientation = 'vertical'  
slider.description = 'Test'
```

Test



50



Перелік атрибутів віджета

slider.keys

```
['_dom_classes',  
'_model_module',  
'_model_module_version',  
'_model_name',  
'_view_count',  
'_view_module',  
'_view_module_version',  
'_view_name',  
'continuous_update',  
'description',  
'description_tooltip',  
'disabled',  
'layout',  
'max',  
'min',  
'orientation',  
'readout',  
'readout_format',  
'step',  
'style',  
'value']
```



Обробка подій

```
number = widgets.IntSlider()
result = widgets.Output()
display(number, result)

def power_of_two(change):
    result.clear_output()
    with result:
        print(f"2^{change['new']} = {2**int(change['new'])}")

number.observe(power_of_two, names='value')
```



2¹⁰ = 1024

Для обробки подій слайдера функції передається спеціальний аргумент **change** (який фактично є словником, ключі якого дають змогу діставатися атрибутів віджетів), а результати зазвичай виводять у спеціальну область, яка задається віджетом **Output**. При цьому ця функція передається методу-спостерігачу **observe** об'єкта слайдера (іменований аргумент **names** у нашому випадку вказує на те, що значеннями словника будуть значення атрибуту **value**):



Приклади віджетів

```
txt = widgets.Text(description = 'What is your name?',  
                    style = {'description_width': 'initial'})  
display(txt)  
  
def handle_submit(sender):  
    print(f'Hello, {sender.value}!')  
  
txt.on_submit(handle_submit)
```

What is your name?

Hello, Roman!



Приклади віджетів

```
a = widgets.FloatText()  
b = widgets.FloatSlider(step=0.01)  
display(a,b)  
  
mylink = widgets.jslink((a, 'value'), (b, 'value'))
```

36,04



36.04



Приклади віджетів

```
widgets.RadioButtons(  
    options=['pepperoni', 'pineapple', 'anchovies'],  
    value='pineapple',  
    description='Pizza topping:',  
    style = {'description_width': 'initial'},  
    disabled=False  
)
```

Pizza topping: ☐ pepperoni
☒ pineapple
☐ anchovies



Віджет **interact**

```
def power_of_two(x):  
    return f"2^{x} = {2**x}"  
  
widgets.interact(power_of_two, x=10)
```

x  10

'2^10 = 1024'

Використовуючи універсальний віджет **interact**, можна отримувати результат одразу після зміни значення, не запускаючи щоразу комірку на виконання

```
def power_of_two(x):  
    return f"2^{x} = {2**x}"  
  
widgets.interact(power_of_two, x=(-10, 10))
```

x  -1

'2^-1 = 0.5'

```
def power_of_two(x):  
    return f"2^{x} = {2**x}"  
  
widgets.interact(power_of_two, x=[1, 2, 3])
```

x  3

'2^3 = 8'



Інтерактивна графіка

```
from ipywidgets import interactive
import math
```

```
def plotter(a, b): #  $f(x) = ax \sin(bx)$ 
    x = [i/1000 for i in range(1, 1001)]
    y = [a * i * math.sin(b*i) for i in x]
    plt.plot(x, y)
    plt.ylim(-3, 3)
    plt.show()
```

```
interactive(plotter, a=[-1, 0, 1], b=100)
```

